

07_text_content_indicator_construction

May 20, 2026

1 07 Text Content Indicator Construction

```
[1]: from pathlib import Path
import re

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

pd.set_option('display.max_columns', 200)
pd.set_option('display.width', 180)

PROJECT_ROOT = Path.cwd()
if PROJECT_ROOT.name == 'notebooks':
    PROJECT_ROOT = PROJECT_ROOT.parent

DATA_DIR = PROJECT_ROOT / 'data'
CORPUS_DIR = DATA_DIR / 'corpus'
INDICATORS_DIR = DATA_DIR / 'indicators'
FIGURES_DIR = PROJECT_ROOT / 'figures' / '07_notebook_figures'

INDICATORS_DIR.mkdir(parents=True, exist_ok=True)
FIGURES_DIR.mkdir(parents=True, exist_ok=True)

MASTER_PARQUET = CORPUS_DIR / 'master_articles.parquet'
MASTER_CSV = CORPUS_DIR / 'master_articles_light.csv'

ARTICLE_OUT_CSV = INDICATORS_DIR / 'article_text_content_indicators.csv'
ARTICLE_OUT_PARQUET = INDICATORS_DIR / 'article_text_content_indicators.parquet'
MONTHLY_OUT_CSV = INDICATORS_DIR / 'monthly_text_content_indicators.csv'
MONTHLY_OUT_PARQUET = INDICATORS_DIR / 'monthly_text_content_indicators.parquet'
EVENT_SUMMARY_OUT_CSV = INDICATORS_DIR / 'text_indicator_event_window_summary.
↳csv'

SHOCKS = {
    'Cologne': pd.Timestamp('2016-01-01'),
    'Chemnitz': pd.Timestamp('2018-08-01'),
```

```
'Correctiv': pd.Timestamp('2024-01-01'),
}
```

```
[2]: def load_master() -> pd.DataFrame:
    if MASTER_PARQUET.exists():
        return pd.read_parquet(MASTER_PARQUET)
    if MASTER_CSV.exists():
        return pd.read_csv(MASTER_CSV)
    raise FileNotFoundError(f'Missing master corpus file: {MASTER_PARQUET} and
    ↳ {MASTER_CSV}')

master = load_master().copy()

required_input_cols = ['keep_drop_review', 'date', 'month', 'article_id',
    ↳ 'text_body']
missing_input = [c for c in required_input_cols if c not in master.columns]
if missing_input:
    raise ValueError(f'Master corpus missing required columns: {missing_input}')

kept = master[master['keep_drop_review'].astype(str).str.lower() == 'keep'].
    ↳ copy()
if kept.empty:
    raise ValueError('Kept corpus is empty after keep_drop_review filter.')

if kept['article_id'].isna().any():
    raise ValueError('article_id has missing values in kept corpus.')
if kept['article_id'].duplicated().any():
    dupes = kept.loc[kept['article_id'].duplicated(), 'article_id'].head(10).
    ↳ tolist()
    raise ValueError(f'Found duplicate article_id values in kept corpus:
    ↳ {dupes}')

kept['date'] = pd.to_datetime(kept['date'], errors='coerce')
if kept['date'].isna().any():
    raise ValueError('Date parsing failed for some kept articles.')

kept['month_dt'] = kept['date'].dt.to_period('M').dt.to_timestamp()
kept['year_month'] = kept['month_dt'].dt.strftime('%Y-%m')
kept['year'] = kept['month_dt'].dt.year
kept['month_num'] = kept['month_dt'].dt.month

if kept['text_body'].fillna('').str.strip().eq('').mean() > 0.1:
    raise ValueError('More than 10% of kept articles have empty text_body.')
```

```
[3]: GERMAN_CHAR_MAP = str.maketrans({
    "ä": "ae", "ö": "oe", "ü": "ue", "ß": "ss",
    "Ä": "ae", "Ö": "oe", "Ü": "ue",
```

```

})

def normalize_german_ascii(text: str) -> str:
    if text is None:
        return ''
    s = str(text).translate(GERMAN_CHAR_MAP).lower()
    s = re.sub(r'[^a-z0-9\s]', ' ', s)
    s = re.sub(r'\s+', ' ', s).strip()
    return s

_umlaut_probe = normalize_german_ascii('Flüchtlinge äußern größere Sorge über
↳Abschiebung')
assert 'fluechtlinge' in _umlaut_probe
assert 'aeussern' in _umlaut_probe
assert 'groessere' in _umlaut_probe
assert 'ueber' in _umlaut_probe
assert 'auslaender' in normalize_german_ascii('Ausländer')
assert 'uebergreif' in normalize_german_ascii('Übergriff')

TERM_PATTERNS = {
    'migration': [
        r'\bmigrat\w*\b', r'\bmigrant\w*\b', r'\bfluechtling\w*\b',
↳r'\bgefluechtet\w*\b',
        r'\basyl\w*\b', r'\bzuwander\w*\b', r'\beinwander\w*\b',
↳r'\bauslaender\w*\b',
        r'\bauslaendisch\w*\b', r'\babschieb\w*\b', r'\bdeport\w*\b',
↳r'\bintegrat\w*\b',
        r'\bgrenz\w*\b', r'\bschutzsuch\w*\b', r'\bunterkunft\w*\b',
↳r'\bfluechtlingsheim\w*\b',
        r'\baufnahm\w*\b'
    ],
    'security': [
        r'\bkriminal\w*\b', r'\bstrafat\w*\b', r'\bstrafataeter\w*\b',
↳r'\btatverdaechtig\w*\b',
        r'\bverdaechtig\w*\b', r'\bpolizei\w*\b', r'\bsicherheit\w*\b',
↳r'\bgewalt\w*\b',
        r'\bmesser\w*\b', r'\bmord\w*\b', r'\btotschlag\w*\b',
↳r'\bvergewaltig\w*\b',
        r'\bsexuell\w*\b', r'\bbelaestig\w*\b', r'\buebergreif\w*\b',
↳r'\bausschreit\w*\b',
        r'\brandal\w*\b', r'\bkrawall\w*\b', r'\bmob\w*\b', r'\bhetzjagd\w*\b',
        r'\bdemonstrat\w*\b', r'\bprotest\w*\b', r'\bordnung\w*\b',
↳r'\brechtsstaat\w*\b',
        r'\binnere\s+sicherheit\b'
    ],
    'afd_far_right': [

```

```

        r'\bafd\b', r'\balternative\s+fuer\s+deutschland\b',␣
↪r'\brechtspopul\w*\b',
        r'\brechtsextrem\w*\b', r'\brechte\w*\b', r'\brechtsradikal\w*\b',␣
↪r'\brechtsruck\w*\b',
        r'\bidentitaer\w*\b', r'\bpegida\w*\b', r'\bhoecke\w*\b',␣
↪r'\bweidel\w*\b',
        r'\bgauland\w*\b', r'\bchrupalla\w*\b', r'\bnationalistisch\w*\b',␣
↪r'\bvoelkisch\w*\b'
    ],
    'remigration_democracy': [
        r'\bremigration\w*\b', r'\bgeheimtreffen\w*\b', r'\bpotsdam\w*\b',␣
↪r'\bcorrectiv\w*\b',
        r'\bdeport\w*\b', r'\bvertreib\w*\b', r'\bmasterplan\w*\b',␣
↪r'\bidentitaer\w*\s+beweg\w*\b',
        r'\bsellner\w*\b', r'\brechtsextremismus\w*\b', r'\bdemokrati\w*\b',␣
↪r'\bverfassungsfeind\w*\b',
        r'\bverfassungsschutz\w*\b', r'\bparteiverbot\w*\b',␣
↪r'\bmenschenwuerd\w*\b', r'\bmassendeport\w*\b'
    ],
    'negative_threat': [
        r'\bangst\w*\b', r'\bsorg\w*\b', r'\bgefahr\w*\b',␣
↪r'\bgefaehrlich\w*\b', r'\bbedroh\w*\b',
        r'\bkris\w*\b', r'\beskalat\w*\b', r'\bchaos\w*\b', r'\bproblem\w*\b',␣
↪r'\bkonflikt\w*\b',
        r'\bgewalt\w*\b', r'\bhass\w*\b', r'\bhetz\w*\b',␣
↪r'\bextremismus\w*\b', r'\bradikal\w*\b',
        r'\bskandal\w*\b', r'\bschock\w*\b', r'\bempoer\w*\b', r'\bwut\w*\b',␣
↪r'\billegal\w*\b',
        r'\bmissbrauch\w*\b', r'\bueberforderung\w*\b', r'\bscheiter\w*\b',␣
↪r'\bversag\w*\b'
    ],
}

```

```

COMPILED = {k: re.compile(''.join(v)) for k, v in TERM_PATTERNS.items()}

```

```

def count_group_terms(norm_text: str) -> dict:
    out = {}
    for group, patt in COMPILED.items():
        out[f'{group}_terms_count'] = len(patt.findall(norm_text))
    return out

```

```

kept['text_normalized'] = kept['text_body'].fillna('').

```

```

↪map(normalize_german_ascii)

```

```

kept['text_token_count'] = kept['text_normalized'].str.split().map(len)

```

```

wc = pd.to_numeric(kept.get('word_count', np.nan), errors='coerce')

```

```

kept['word_count_used'] = wc.where(wc > 0, kept['text_token_count'])
kept['word_count_used'] = kept['word_count_used'].
    ↪ fillna(kept['text_token_count'])

count_df = kept['text_normalized'].map(count_group_terms).apply(pd.Series)
article = pd.concat([kept, count_df], axis=1)

for group in TERM_PATTERNS.keys():
    c = f'{group}_terms_count'
    p = f'{group}_terms_per_1000_words'
    h = f'has_{group}_terms'
    article[p] = np.where(article['word_count_used'] > 0, article[c] * 1000.0 /
    ↪ article['word_count_used'], np.nan)
    article[h] = (article[c] > 0).astype(int)

article['security_share_of_migration_context'] = np.where(
    article['migration_terms_count'] > 0,
    article['security_terms_count'] / article['migration_terms_count'],
    np.nan,
)
article['afd_mentions_binary'] = article['has_afd_far_right_terms']
article['remigration_democracy_binary'] =
    ↪ article['has_remigration_democracy_terms']
article['negative_threat_intensity'] =
    ↪ article['negative_threat_terms_per_1000_words']

all_term_sum = article[[f'{g}_terms_count' for g in TERM_PATTERNS.keys()]].
    ↪ sum().sum()
if all_term_sum == 0:
    raise ValueError('All text indicator term counts are zero. Dictionary
    ↪ likely failed.')

base_cols = [
    'article_id', 'date', 'year_month', 'month_dt', 'year', 'month_num',
    'publication', 'publication_normalized', 'word_count', 'body_char_count',
    ↪ 'word_count_used',
    'corpus_type', 'corpus_name', 'batch_id', 'source_archive'
]
existing_base = [c for c in base_cols if c in article.columns]

indicator_cols = []
for g in TERM_PATTERNS.keys():
    indicator_cols.extend([
        f'{g}_terms_count',
        f'{g}_terms_per_1000_words',
        f'has_{g}_terms',
    ])

```

```

    ])
indicator_cols.extend([
    'security_share_of_migration_context',
    'afd_mentions_binary',
    'remigration_democracy_binary',
    'negative_threat_intensity',
])

article_out = article[existing_base + indicator_cols].copy()
article_out = article_out.sort_values(['month_dt', 'article_id']).
    ↪reset_index(drop=True)

if len(article_out) != len(kept):
    raise ValueError(f'Article output row count mismatch: {len(article_out)} vs_
    ↪kept {len(kept)}')

for c in [col for col in article_out.columns if col.
    ↪endswith('_per_1000_words')]:
    bad = (~np.isfinite(article_out[c])) & article_out[c].notna()
    if bad.any():
        raise ValueError(f'Found non-finite values in {c}')

article_out.to_csv(ARTICLE_OUT_CSV, index=False)
article_out.to_parquet(ARTICLE_OUT_PARQUET, index=False)

for g in TERM_PATTERNS.keys():
    print(g, int(article_out[f'{g}_terms_count'].sum()))

```

```

migration 139631
security 25842
afd_far_right 11942
remigration_democracy 6319
negative_threat 24645

```

```

[4]: agg_dict = {
    'month_dt': ('month_dt', 'first'),
    'year': ('year', 'first'),
    'month_num': ('month_num', 'first'),
    'n_articles_text_indicators': ('article_id', 'size'),
    'mean_word_count': ('word_count_used', 'mean'),
    'median_word_count': ('word_count_used', 'median'),
    'total_word_count': ('word_count_used', 'sum'),
    'security_share_of_migration_context_mean':
    ↪('security_share_of_migration_context', 'mean'),
}

for g in TERM_PATTERNS.keys():

```

```

agg_dict[f'{g}_terms_per_1000_mean'] = (f'{g}_terms_per_1000_words', 'mean')
agg_dict[f'{g}_terms_per_1000_median'] = (f'{g}_terms_per_1000_words',
↳ 'median')
agg_dict[f'share_articles_with_{g}_terms'] = (f'has_{g}_terms', 'mean')
agg_dict[f'{g}_terms_total'] = (f'{g}_terms_count', 'sum')

monthly = article_out.groupby('year_month', as_index=False).agg(**agg_dict)
monthly = monthly.sort_values('month_dt').reset_index(drop=True)

expected_range = pd.period_range('2013-01', '2025-12', freq='M')
actual_range = pd.PeriodIndex(monthly['month_dt'].dt.to_period('M').unique(),
↳ freq='M')
missing_months_before_fill = expected_range.difference(actual_range)

full_months = pd.DataFrame({'year_month': expected_range.strftime('%Y-%m')})
full_months['month_dt'] = expected_range.to_timestamp()
full_months['year'] = full_months['month_dt'].dt.year
full_months['month_num'] = full_months['month_dt'].dt.month

monthly_out = full_months.merge(monthly.drop(columns=['month_dt', 'year',
↳ 'month_num'], errors='ignore'), on='year_month', how='left')
monthly_out['month'] = monthly_out['year_month']

zero_fill_cols = ['n_articles_text_indicators', 'total_word_count'] +
↳ [f'{g}_terms_total' for g in TERM_PATTERNS.keys()]
for col in zero_fill_cols:
    if col in monthly_out.columns:
        monthly_out[col] = monthly_out[col].fillna(0)

for col in ['n_articles_text_indicators', 'total_word_count'] +
↳ [f'{g}_terms_total' for g in TERM_PATTERNS.keys()]:
    if col in monthly_out.columns:
        monthly_out[col] = monthly_out[col].astype(int)

monthly_out['text_indicator_month_has_articles'] =
↳ monthly_out['n_articles_text_indicators'] > 0

front_cols = [
    'month', 'year_month', 'month_dt', 'year', 'month_num',
    'text_indicator_month_has_articles',
    'n_articles_text_indicators', 'mean_word_count', 'median_word_count',
↳ 'total_word_count',
    'migration_terms_per_1000_mean', 'migration_terms_total',
↳ 'share_articles_with_migration_terms',
    'security_terms_per_1000_mean', 'security_terms_total',
↳ 'share_articles_with_security_terms',

```

```

        'afd_far_right_terms_per_1000_mean', 'afd_far_right_terms_total',
        ↪ 'share_articles_with_afd_far_right_terms',
        'remigration_democracy_terms_per_1000_mean',
        ↪ 'remigration_democracy_terms_total',
        ↪ 'share_articles_with_remigration_democracy_terms',
        'negative_threat_terms_per_1000_mean', 'negative_threat_terms_total',
        ↪ 'share_articles_with_negative_threat_terms',
        'security_share_of_migration_context_mean',
    ]
existing_front = [c for c in front_cols if c in monthly_out.columns]
other_cols = [c for c in monthly_out.columns if c not in existing_front]
monthly_out = monthly_out[existing_front + other_cols].copy()
monthly_out = monthly_out.sort_values('month_dt').reset_index(drop=True)

if monthly_out['year_month'].duplicated().any():
    raise ValueError('Monthly indicators have duplicate months.')
if len(monthly_out) != len(expected_range):
    raise ValueError(f'Monthly output length mismatch: {len(monthly_out)} vs
    ↪ expected {len(expected_range)}')

missing_months_after_fill = expected_range.difference(pd.
    ↪ PeriodIndex(monthly_out['month_dt'].dt.to_period('M').unique(), freq='M'))
if len(missing_months_after_fill) > 0:
    raise ValueError(f'Monthly output still missing months after fill: {[m.
    ↪ strftime("%Y-%m") for m in missing_months_after_fill]}')

monthly_out.to_csv(MONTHLY_OUT_CSV, index=False)
monthly_out.to_parquet(MONTHLY_OUT_PARQUET, index=False)

check_cols = [
    'year_month', 'n_articles_text_indicators',
    'migration_terms_per_1000_mean', 'security_terms_per_1000_mean',
    'afd_far_right_terms_per_1000_mean',
    ↪ 'remigration_democracy_terms_per_1000_mean',
    'negative_threat_terms_per_1000_mean', 'text_indicator_month_has_articles',
]
check_cols = [c for c in check_cols if c in monthly_out.columns]

ids_201908 = set(article_out.loc[article_out['year_month'] == '2019-08',
    ↪ 'article_id'])
ids_201909 = set(article_out.loc[article_out['year_month'] == '2019-09',
    ↪ 'article_id'])
assert len(ids_201909) > 0, '2019-09 has no article indicators after refresh'
assert ids_201908 != ids_201909, '2019-09 article IDs are identical to 2019-08;
    ↪ possible duplicate archive issue remains'

```



```

[5]: def add_shock_lines(ax):
    for label, dt in SHOCKS.items():
        ax.axvline(dt, linestyle='--', linewidth=1.1, alpha=0.85,
        ↪label=f'{label} ({dt.strftime("%Y-%m")})')

plot_specs = [
    ('n_articles_text_indicators', 'Monthly Article Count Used for Text_
    ↪Indicators', 'n_articles_text_indicators',
    ↪'monthly_n_articles_text_indicators.png'),
    ('migration_terms_per_1000_mean', 'Migration Terms per 1000 Words (Monthly_
    ↪Mean)', 'migration_terms_per_1000_mean',
    ↪'monthly_migration_terms_per_1000_mean.png'),
    ('security_terms_per_1000_mean', 'Security/Public-Order Terms per 1000_
    ↪Words (Monthly Mean)', 'security_terms_per_1000_mean',
    ↪'monthly_security_terms_per_1000_mean.png'),
    ('afd_far_right_terms_per_1000_mean', 'AfD/Far-Right Terms per 1000 Words_
    ↪(Monthly Mean)', 'afd_far_right_terms_per_1000_mean',
    ↪'monthly_afd_far_right_terms_per_1000_mean.png'),
    ('remigration_democracy_terms_per_1000_mean', 'Remigration/Democracy-Threat_
    ↪Terms per 1000 Words (Monthly Mean)',
    ↪'remigration_democracy_terms_per_1000_mean',
    ↪'monthly_remigration_democracy_terms_per_1000_mean.png'),
    ('negative_threat_terms_per_1000_mean', 'Negative/Threat Terms per 1000_
    ↪Words (Monthly Mean)', 'negative_threat_terms_per_1000_mean',
    ↪'monthly_negative_threat_terms_per_1000_mean.png'),
]

for col, title, ylab, fname in plot_specs:
    fig, ax = plt.subplots(figsize=(12, 4.8))
    ax.plot(monthly_out['month_dt'], monthly_out[col], linewidth=1.8)
    add_shock_lines(ax)
    ax.set_title(title)
    ax.set_xlabel('Month')
    ax.set_ylabel(ylab)
    ax.grid(alpha=0.25)
    ax.legend(loc='upper left', ncol=2)
    fig.tight_layout()
    out = FIGURES_DIR / fname
    fig.savefig(out, dpi=150)
    plt.show()
    print('Saved:', out)

std_cols = [
    'migration_terms_per_1000_mean',
    'security_terms_per_1000_mean',

```

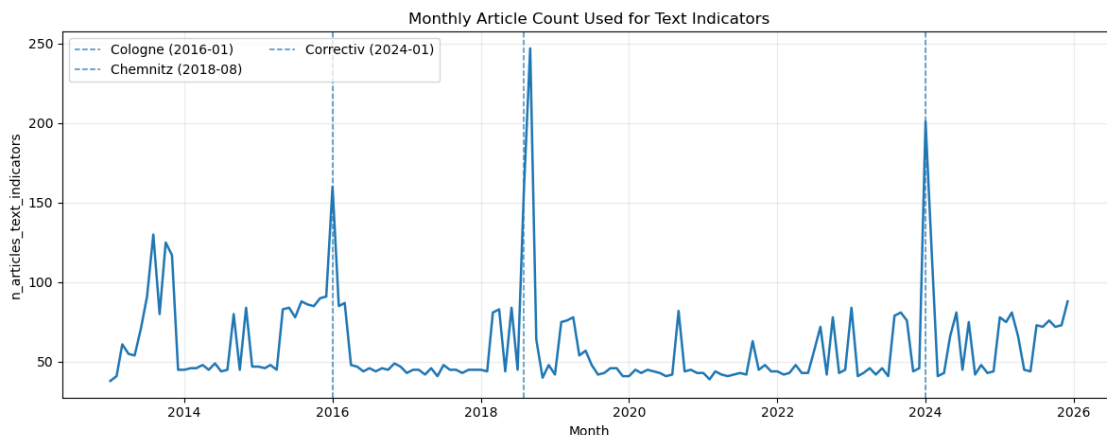
```

    'remigration_democracy_terms_per_1000_mean',
    'negative_threat_terms_per_1000_mean',
]

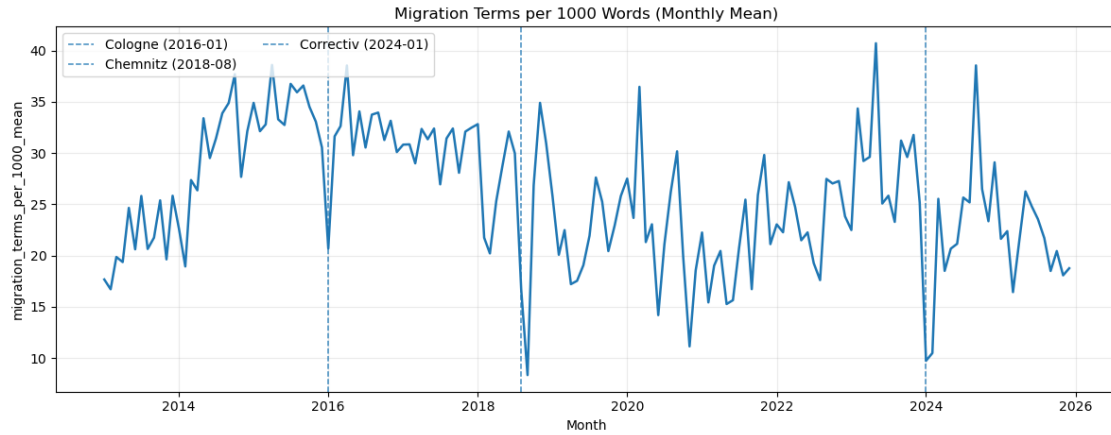
std_df = monthly_out[['month_dt'] + std_cols].copy()
for c in std_cols:
    s = pd.to_numeric(std_df[c], errors='coerce')
    std_df[c + '_z'] = (s - s.mean()) / (s.std(ddof=0) if s.std(ddof=0) != 0
    else 1.0)

fig, ax = plt.subplots(figsize=(12, 5))
for c, color in zip(std_cols, ['#1f4e79', '#7a2e2e', '#2f5d50', '#5a4a78']):
    ax.plot(std_df['month_dt'], std_df[c + '_z'], linewidth=1.7, label=c,
    color=color)
add_shock_lines(ax)
ax.set_title('Standardized Monthly Text Indicators (Descriptive)')
ax.set_xlabel('Month')
ax.set_ylabel('z-score')
ax.grid(alpha=0.25)
ax.legend(loc='upper left', ncol=2)
fig.tight_layout()
out = FIGURES_DIR / 'combined_standardized_text_indicators.png'
fig.savefig(out, dpi=150)
plt.show()

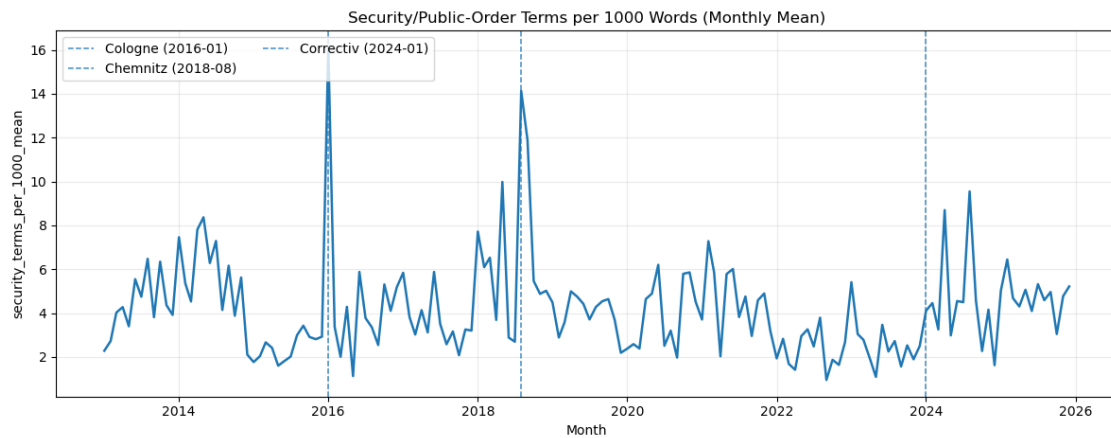
```



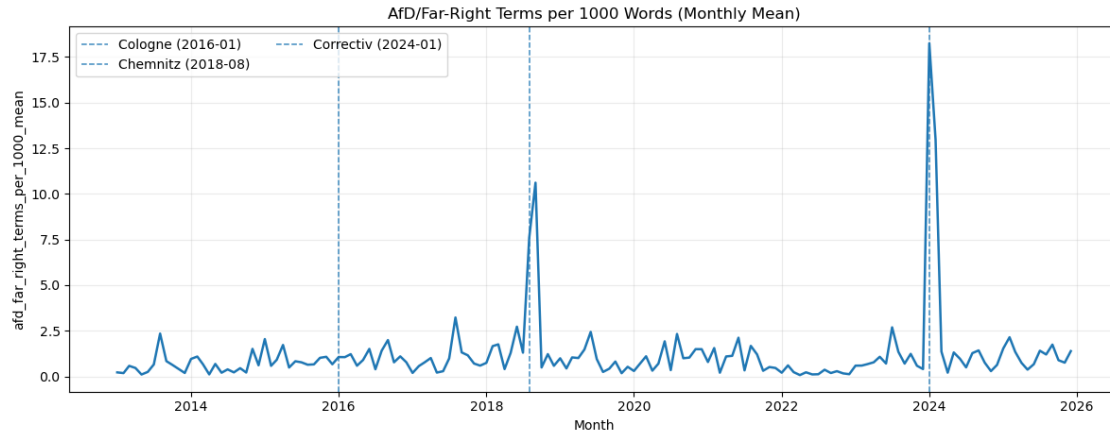
Saved: c:\PythonProjects\AfD-project\figures\07_notebook_figures\monthly_n_articles_text_indicators.png



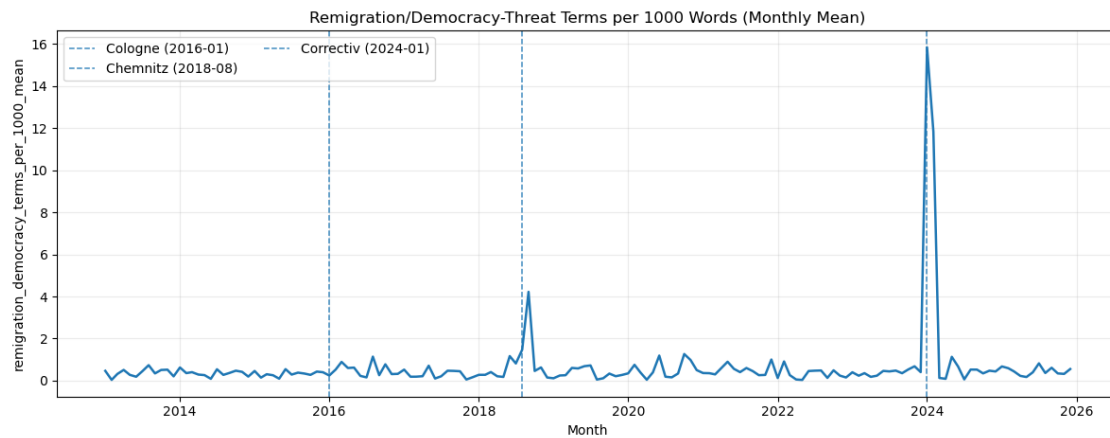
Saved: c:\PythonProjects\AfD-project\figures\07_notebook_figures\monthly_migration_terms_per_1000_mean.png



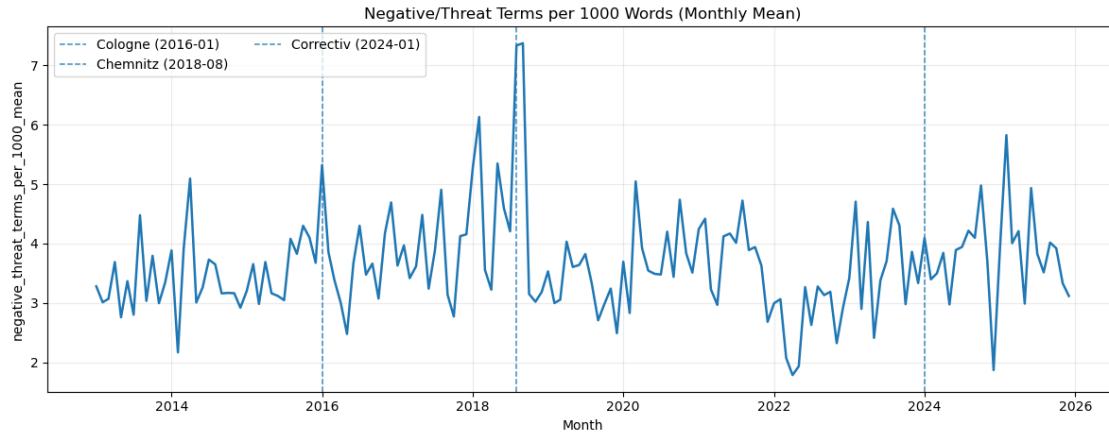
Saved: c:\PythonProjects\AfD-project\figures\07_notebook_figures\monthly_security_terms_per_1000_mean.png



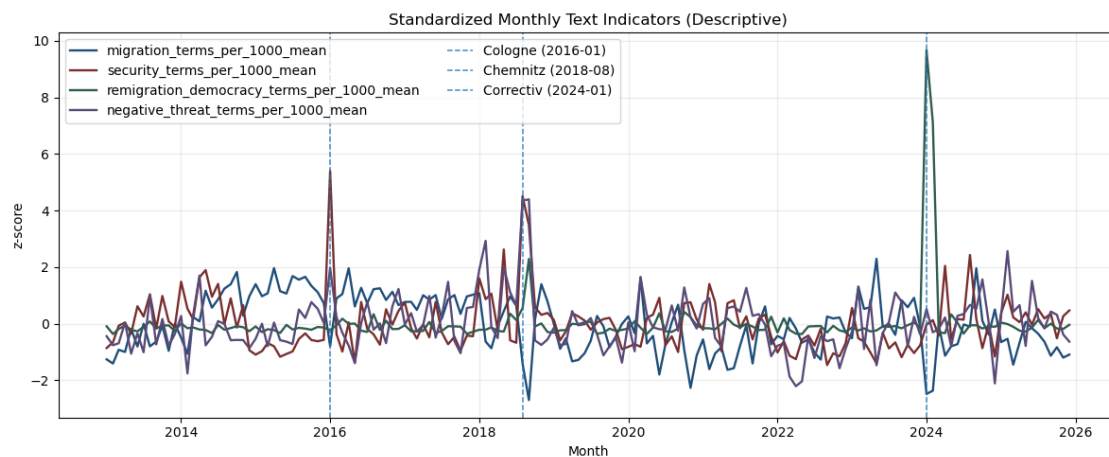
Saved: c:\PythonProjects\AfD-project\figures\07_notebook_figures\monthly_afd_far_right_terms_per_1000_mean.png



Saved: c:\PythonProjects\AfD-project\figures\07_notebook_figures\monthly_remigration_democracy_terms_per_1000_mean.png



Saved: c:\PythonProjects\AfD-project\figures\07_notebook_figures\monthly_negative_threat_terms_per_1000_mean.png



```
[6]: selected_cols = [
    'n_articles_text_indicators',
    'migration_terms_per_1000_mean',
    'security_terms_per_1000_mean',
    'afd_far_right_terms_per_1000_mean',
    'remigration_democracy_terms_per_1000_mean',
    'negative_threat_terms_per_1000_mean',
]

for label, shock_month in SHOCKS.items():
    w = monthly_out[(monthly_out['month_dt'] >= shock_month - pd.
        ↳ DateOffset(months=6)) & (monthly_out['month_dt'] <= shock_month + pd.
        ↳ DateOffset(months=6))].copy()
```

```

    w['rel_month'] = ((w['month_dt'].dt.year - shock_month.year) * 12 +
↳(w['month_dt'].dt.month - shock_month.month)).astype(int)
    w = w.sort_values('month_dt')
    print('Shock +/-6:', label, shock_month.strftime('%Y-%m'))
    print(w[['year_month', 'rel_month'] + selected_cols].to_string(index=False))
    print('')

summary_rows = []
for label, shock_month in SHOCKS.items():
    for win in [3, 6]:
        w = monthly_out[(monthly_out['month_dt'] >= shock_month - pd.
↳DateOffset(months=win)) & (monthly_out['month_dt'] <= shock_month + pd.
↳DateOffset(months=win))].copy()
        w['rel_month'] = ((w['month_dt'].dt.year - shock_month.year) * 12 +
↳(w['month_dt'].dt.month - shock_month.month)).astype(int)
        pre = w[w['rel_month'] < 0]
        post = w[w['rel_month'] > 0]
        shock_row = w[w['rel_month'] == 0]
        for ind in selected_cols:
            summary_rows.append({
                'shock_label': label,
                'shock_month': shock_month.strftime('%Y-%m'),
                'indicator': ind,
                'window_months': win,
                'pre_mean': pre[ind].mean(),
                'post_mean': post[ind].mean(),
                'change_post_minus_pre': post[ind].mean() - pre[ind].mean(),
                'shock_month_value': shock_row[ind].iloc[0] if len(shock_row)
↳== 1 else np.nan,
                'n_months_pre': len(pre),
                'n_months_post': len(post),
            })

event_summary = pd.DataFrame(summary_rows)
event_summary.to_csv(EVENT_SUMMARY_OUT_CSV, index=False)

```

Shock +/-6: Cologne 2016-01

year_month	rel_month	n_articles_text_indicators	migration_terms_per_1000_mean	security_terms_per_1000_mean	afd_far_right_terms_per_1000_mean	remigration_democracy_terms_per_1000_mean	negative_threat_terms_per_1000_mean
2015-07	-6		78	36.744290			
2.019100		0.774020					
0.287608		3.045782					
2015-08	-5		88	35.924009			
2.989692		0.641663					
0.380044		4.079031					
2015-09	-4		86	36.582634			

3.428024		0.661289		
0.337481		3.826830		
2015-10	-3		85	34.504662
2.912465		1.019384		
0.274491		4.300402		
2015-11	-2		90	33.059745
2.812118		1.075311		
0.430070		4.101326		
2015-12	-1		91	30.541767
2.923229		0.668327		
0.398917		3.676347		
2016-01	0		160	20.690986
16.113945		1.061618		
0.244027		5.323650		
2016-02	1		85	31.628857
3.375920		1.057194		
0.524713		3.855343		
2016-03	2		87	32.624852
2.008039		1.220516		
0.890255		3.383951		
2016-04	3		48	38.535137
4.288379		0.587285		
0.600885		2.995970		
2016-05	4		47	29.783843
1.127660		0.900754		
0.618695		2.476253		
2016-06	5		44	34.058535
5.879610		1.509385		
0.224398		3.668425		
2016-07	6		46	30.536024
3.776114		0.393306		
0.154813		4.299850		

Shock +/-6: Chemnitz 2018-08

year_month	rel_month	n_articles_text_indicators	migration_terms_per_1000_mean	security_terms_per_1000_mean	afd_far_right_terms_per_1000_mean	remigration_democracy_terms_per_1000_mean	negative_threat_terms_per_1000_mean
2018-02	-6		44				21.739395
6.094171		1.659223					
0.276197		6.132502					
2018-03	-5		81				20.209753
6.528591		1.757668					
0.407601		3.557194					
2018-04	-4		83				25.305153
3.688403		0.401600					
0.207000		3.222521					
2018-05	-3		44				28.712573
9.979395		1.289397					

0.176565			5.349288	
2018-06	-2		84	32.093096
2.885574			2.725183	
1.167407			4.594193	
2018-07	-1		45	29.965239
2.696084			1.301466	
0.816104			4.206995	
2018-08	0		158	16.697127
14.128618			7.604621	
1.472552			7.337937	
2018-09	1		247	8.346201
11.872023			10.611414	
4.220916			7.374902	
2018-10	2		64	26.859455
5.464897			0.491198	
0.459709			3.150977	
2018-11	3		40	34.892491
4.876096			1.221995	
0.627184			3.020050	
2018-12	4		48	30.947139
5.019104			0.582579	
0.148705			3.182414	
2019-01	5		42	25.809626
4.487241			0.993031	
0.111622			3.530687	
2019-02	6		75	20.078653
2.888916			0.437168	
0.244125			2.997754	

Shock +/-6: Correctiv 2024-01

year_month	rel_month	n_articles_text_indicators	migration_terms_per_1000_mean	security_terms_per_1000_mean	afd_far_right_terms_per_1000_mean	remigration_democracy_terms_per_1000_mean	negative_threat_terms_per_1000_mean
2023-07	-6		41				25.836879
2.251177			2.688887				
0.438113			3.703873				
2023-08	-5		79				23.284618
2.725435			1.348609				
0.477407			4.586849				
2023-09	-4		81				31.212619
1.568108			0.704221				
0.352645			4.307351				
2023-10	-3		76				29.607353
2.530270			1.235712				
0.529612			2.978377				
2023-11	-2		44				31.764793
1.894647			0.584137				
0.681364			3.861029				

2023-12	-1		46	25.166726
2.499861		0.403373		
0.404035		3.334212		
2024-01	0		201	9.742082
4.108073		18.249246		
15.842675		4.092657		
2024-02	1		117	10.489429
4.458344		12.966649		
11.836392		3.396432		
2024-03	2		41	25.532084
3.256880		1.361379		
0.127617		3.497977		
2024-04	3		43	18.509327
8.696476		0.204800		
0.086731		3.842648		
2024-05	4		66	20.674007
2.983241		1.313263		
1.132032		2.973877		
2024-06	5		81	21.148360
4.550623		0.964112		
0.666840		3.890751		
2024-07	6		45	25.657812
4.496735		0.496943		
0.059551		3.943956		

[7]:

```
focus_inds = [
    'migration_terms_per_1000_mean',
    'security_terms_per_1000_mean',
    'afd_far_right_terms_per_1000_mean',
    'remigration_democracy_terms_per_1000_mean',
    'negative_threat_terms_per_1000_mean',
]

summary_focus = event_summary[(event_summary['window_months'] == 6) &
    ↪(event_summary['indicator'].isin(focus_inds))].copy()
summary_focus = summary_focus.sort_values(['shock_label', 'indicator'])
```

1.1 Interpretation

Correctiv can be better reflected in remigration, democracy-threat terms Chemnitz may be better reflected in security/public-order and far-right indicators Cologne shows migration/security/negative-threat